Lecture 27: Practice Problems Master Solutions & Engineering Systems Manual

CreditWise Loan Approval System (Minor Project)

Complete, rigorous mathematical derivations, production code implementations, and systems case studies.

Problem 01: Automated Credit Data Auditor & Statistical Imputer (CreditRiskDataAuditor)

💡 Production Python Implementation

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import SimpleImputer

class CreditRiskDataAuditor(BaseEstimator, TransformerMixin):
    """
    Automated data auditor and statistical imputer for retail credit risk datasets
    Applies Mean imputation to continuous features and Mode imputation to categori
    """
    def __init__(self):
        self.num_imputer = SimpleImputer(strategy='mean')
        self.cat_imputer = SimpleImputer(strategy='most_frequent')
        self.num_cols_ = None
        self.cat_cols_ = None

    def fit(self, X, y=None):
        X_df = pd.DataFrame(X).copy()
        self.num_cols_ = X_df.select_dtypes(include=[np.number]).columns.tolist()
        self.cat_cols_ = X_df.select_dtypes(exclude=[np.number]).columns.tolist()

        if self.num_cols_:
            self.num_imputer.fit(X_df[self.num_cols_])
        if self.cat_cols_:
            self.cat_imputer.fit(X_df[self.cat_cols_])
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X).copy()
```

```
        if self.num_cols_:
            X_df[self.num_cols_] = self.num_imputer.transform(X_df[self.num_cols_]
        if self.cat_cols_:
            X_df[self.cat_cols_] = self.cat_imputer.transform(X_df[self.cat_cols_]
        return X_df

# Unit Test
auditor = CreditRiskDataAuditor()
df_raw = pd.DataFrame({
    'Income': [50000.0, np.nan, 75000.0, 30000.0],
    'Status': ['Salaried', 'Self-Employed', np.nan, 'Salaried']
})
df_clean = auditor.fit_transform(df_raw)
assert df_clean.isna().sum().sum() == 0, 'Null values must be completely eliminate
assert df_clean.loc[1, 'Income'] == 51666.666666666664 or round(df_clean.loc[1, 'I
print('Problem 01 Passed!')
```

Problem 02: Robust Financial Categorical Feature Encoder (FinancialFeatureEncoder)

💡 Production Python Implementation

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import OneHotEncoder

class FinancialFeatureEncoder(BaseEstimator, TransformerMixin):
    def __init__(self, categorical_cols=None):
        self.categorical_cols = categorical_cols
        self.ohe = OneHotEncoder(drop='first', sparse_output=False, handle_unknown
        self.feature_names_out_ = None

    def fit(self, X, y=None):
        X_df = pd.DataFrame(X)
        if self.categorical_cols is None:
            self.categorical_cols = X_df.select_dtypes(exclude=[np.number]).column
        if self.categorical_cols:
            self.ohe.fit(X_df[self.categorical_cols])
            self.feature_names_out_ = self.ohe.get_feature_names_out(self.categori
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X).copy()
        if not self.categorical_cols:
            return X_df
        encoded_arr = self.ohe.transform(X_df[self.categorical_cols])
```

```
        encoded_df = pd.DataFrame(encoded_arr, columns=self.feature_names_out_, in
        remaining_df = X_df.drop(columns=self.categorical_cols)
        return pd.concat([remaining_df, encoded_df], axis=1)

# Unit Test
encoder = FinancialFeatureEncoder(categorical_cols=['Purpose', 'Area'])
sample_train = pd.DataFrame({
    'Amount': [10000, 20000, 15000],
    'Purpose': ['Home', 'Personal', 'Home'],
    'Area': ['Urban', 'Rural', 'Semiurban']
})
train_enc = encoder.fit_transform(sample_train)
assert 'Purpose_Personal' in train_enc.columns, 'Expected dropped dummy column str
print('Problem 02 Passed! Columns:', train_enc.columns.tolist())
```

Problem 03: Tukey IQR Financial Outlier Capping Transformer (OutlierCappingTransformer)

💡 Production Python Implementation

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class OutlierCappingTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, factor=1.5, numerical_cols=None):
        self.factor = factor
        self.numerical_cols = numerical_cols
        self.fences_ = {}

    def fit(self, X, y=None):
        X_df = pd.DataFrame(X)
        cols = self.numerical_cols if self.numerical_cols else X_df.select_dtypes(
        for col in cols:
            q1 = X_df[col].quantile(0.25)
            q3 = X_df[col].quantile(0.75)
            iqr = q3 - q1
            lower_fence = max(0.0, q1 - self.factor * iqr) # non-negative financia
            upper_fence = q3 + self.factor * iqr
            self.fences_[col] = (lower_fence, upper_fence)
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X).copy()
        for col, (fl, fu) in self.fences_.items():
            if col in X_df.columns:
                X_df[col] = X_df[col].clip(lower=fl, upper=fu)
```

```
        return X_df

# Unit Test
capper = OutlierCappingTransformer(numerical_cols=['Income'])
df_test = pd.DataFrame({'Income': [10000, 15000, 12000, 14000, 13000, 500000]}) #
capped = capper.fit_transform(df_test)
assert capped['Income'].max() < 500000, 'Outlier must be capped at upper fence'
print('Problem 03 Passed! Upper Fence Cap:', capper.fences_['Income'][1])
```

Problem 04: Domain-Driven Financial Risk Ratio Generator (FinancialRatioGenerator)

💡 Production Python Implementation

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class FinancialRatioGenerator(BaseEstimator, TransformerMixin):
    def __init__(self, drop_raw_collinear=True):
        self.drop_raw_collinear = drop_raw_collinear

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X_df = pd.DataFrame(X).copy()
        # 1. Non-linear polynomials
        if 'DTI_Ratio' in X_df.columns:
            X_df['DTI_Ratio_sq'] = X_df['DTI_Ratio'] ** 2
        if 'Credit_Score' in X_df.columns:
            X_df['Credit_Score_sq'] = X_df['Credit_Score'] ** 2

        # 2. Financial ratios
        app_inc = X_df.get('Applicant_Income', 0.0)
        coapp_inc = X_df.get('Coapplicant_Income', 0.0)
        tot_inc = app_inc + coapp_inc
        X_df['Total_Income'] = tot_inc

        if 'Loan_Amount' in X_df.columns:
            X_df['Loan_to_Income'] = X_df['Loan_Amount'] / (tot_inc + 1.0)
            if 'Collateral_Value' in X_df.columns:
                X_df['Collateral_Coverage'] = X_df['Collateral_Value'] / (X_df['Lc

        # 3. Optional pruning of raw collinear
        if self.drop_raw_collinear:
            cols_to_drop = [c for c in ['DTI_Ratio', 'Credit_Score'] if c in X_df.
```

```
            X_df = X_df.drop(columns=cols_to_drop)

        return X_df

# Unit Test
gen = FinancialRatioGenerator(drop_raw_collinear=True)
sample_df = pd.DataFrame({
    'Applicant_Income': [40000.0],
    'Coapplicant_Income': [10000.0],
    'Credit_Score': [750.0],
    'DTI_Ratio': [0.30],
    'Loan_Amount': [200000.0],
    'Collateral_Value': [400000.0]
})
res = gen.fit_transform(sample_df)
assert res['Total_Income'].iloc[0] == 50000.0
assert res['Collateral_Coverage'].iloc[0] == 400000.0 / 200001.0
assert 'Credit_Score' not in res.columns, 'Raw collinear feature should be pruned'
print('Problem 04 Passed!')
```

Problem 05: Cost-Sensitive Logistic Regression Underwriter (CostSensitiveLogisticRegression)

💡 Production Python Implementation

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, confusion_matrix

class CostSensitiveLogisticRegression:
    def __init__(self, fp_penalty_weight=4.0, random_state=42):
        self.fp_penalty_weight = fp_penalty_weight
        self.random_state = random_state
        self.weights = {0: self.fp_penalty_weight, 1: 1.0}
        self.clf = LogisticRegression(class_weight=self.weights, random_state=self

    def fit(self, X, y):
        self.clf.fit(X, y)
        return self

    def predict(self, X):
        return self.clf.predict(X)

    def predict_proba(self, X):
        return self.clf.predict_proba(X)

# Unit Test with synthetic imbalanced financial dataset
```

```
np.random.seed(42)
X_dummy = np.random.randn(500, 5)
# True default risk depends heavily on feature 0 and 1
y_dummy = ((X_dummy[:, 0] * 1.5 - X_dummy[:, 1] * 2.0 + np.random.randn(500) * 0.5

# 1. Standard Symmetric
std_model = LogisticRegression(random_state=42).fit(X_dummy[:400], y_dummy[:400])
preds_std = std_model.predict(X_dummy[400:])
cm_std = confusion_matrix(y_dummy[400:], preds_std)

# 2. Cost-Sensitive
cs_model = CostSensitiveLogisticRegression(fp_penalty_weight=5.0).fit(X_dummy[:400
preds_cs = cs_model.predict(X_dummy[400:])
cm_cs = confusion_matrix(y_dummy[400:], preds_cs)

fp_std = cm_std[0, 1]
fp_cs = cm_cs[0, 1]
print(f'Standard Model FP: {fp_std} | Cost-Sensitive Model FP: {fp_cs}')
assert fp_cs <= fp_std, 'Cost-sensitive model must minimize False Positives'
print('Problem 05 Passed!')
```

Problem 06: Distance-Weighted KNN Underwriting Engine (KNNUnderwritingEngine)

💡 Production Python Implementation

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

class KNNUnderwritingEngine:
    def __init__(self, n_neighbors=5, metric='minkowski', p=2):
        self.n_neighbors = n_neighbors
        self.clf = KNeighborsClassifier(n_neighbors=n_neighbors, weights='distance
        self.X_train_ = None
        self.y_train_ = None

    def fit(self, X, y):
        self.X_train_ = np.array(X)
        self.y_train_ = np.array(y)
        self.clf.fit(self.X_train_, self.y_train_)
        return self

    def predict(self, X):
        return self.clf.predict(X)

    def predict_proba(self, X):
        return self.clf.predict_proba(X)
```

```
    def explain_applicant(self, x_single):
        x_reshaped = np.array(x_single).reshape(1, -1)
        distances, indices = self.clf.kneighbors(x_reshaped)
        peer_labels = self.y_train_[indices[0]]
        approval_confidence = np.mean(peer_labels)
        return {
            'peer_indices': indices[0].tolist(),
            'peer_distances': distances[0].tolist(),
            'peer_outcomes': peer_labels.tolist(),
            'unweighted_approval_ratio': approval_confidence
        }

# Unit Test
knn_engine = KNNUnderwritingEngine(n_neighbors=3)
X_dummy = np.array([[10, 2], [11, 2], [10, 3], [50, 10], [52, 9], [51, 11]])
y_dummy = np.array([0, 0, 0, 1, 1, 1])
knn_engine.fit(X_dummy, y_dummy)
explanation = knn_engine.explain_applicant([10.5, 2.5])
assert explanation['unweighted_approval_ratio'] == 0.0, 'Applicant should match lo
print('Problem 06 Passed! Nearest Peer Distances:', explanation['peer_distances'])
```

Problem 07: Generative Gaussian Naive Bayes Risk Assessor (GaussianNBRiskAssessor)

💡 Production Python Implementation

```
import numpy as np
from sklearn.naive_bayes import GaussianNB

class GaussianNBRiskAssessor:
    def __init__(self, var_smoothing=1e-8):
        self.var_smoothing = var_smoothing
        self.model = GaussianNB(var_smoothing=self.var_smoothing)

    def fit(self, X, y):
        self.model.fit(X, y)
        return self

    def predict(self, X):
        return self.model.predict(X)

    def predict_proba(self, X):
        return self.model.predict_proba(X)

    def compute_log_odds(self, X):
        probs = self.predict_proba(X)
```

```
        eps = 1e-12
        p1 = np.clip(probs[:, 1], eps, 1.0 - eps)
        log_odds = np.log(p1 / (1.0 - p1))
        return log_odds

# Unit Test
gnb = GaussianNBRiskAssessor()
X_dummy = np.random.randn(200, 4)
y_dummy = (X_dummy[:, 0] > 0.5).astype(int)
gnb.fit(X_dummy, y_dummy)
log_odds = gnb.compute_log_odds(X_dummy[:5])
assert len(log_odds) == 5, 'Must return log odds for each test sample'
assert np.all(np.isfinite(log_odds)), 'Log odds must be numerically finite'
print('Problem 07 Passed! Sample Log Odds:', log_odds[:3])
```

Problem 08: Profit-Maximizing Underwriting Threshold Tuner (OptimalDecisionThresholdTuner)

💡 Production Python Implementation

```
import numpy as np

class OptimalDecisionThresholdTuner:
    def __init__(self, interest_gain=50000.0, default_loss=250000.0, opportunity_l
        self.gain = interest_gain
        self.loss = default_loss
        self.opp = opportunity_loss
        self.optimal_threshold_ = 0.50
        self.max_profit_ = 0.0

    def fit(self, y_true, y_probs):
        thresholds = np.linspace(0.05, 0.95, 91)
        best_profit = -np.inf
        best_tau = 0.50

        for tau in thresholds:
            preds = (y_probs >= tau).astype(int)
            tp = np.sum((y_true == 1) & (preds == 1))
            fp = np.sum((y_true == 0) & (preds == 1))
            fn = np.sum((y_true == 1) & (preds == 0))
            profit = (tp * self.gain) - (fp * self.loss) - (fn * self.opp)

            if profit > best_profit:
                best_profit = profit
                best_tau = tau

        self.optimal_threshold_ = best_tau
```

```
        self.max_profit_ = best_profit
        return self

    def predict(self, y_probs):
        return (y_probs >= self.optimal_threshold_).astype(int)

# Unit Test
np.random.seed(42)
y_true_mock = np.random.choice([0, 1], size=200, p=[0.7, 0.3])
y_probs_mock = np.random.uniform(0.1, 0.9, size=200)

tuner = OptimalDecisionThresholdTuner(interest_gain=50000, default_loss=250000, op
tuner.fit(y_true_mock, y_probs_mock)

# Evaluate profit at 0.50
preds_default = (y_probs_mock >= 0.50).astype(int)
tp_def = np.sum((y_true_mock == 1) & (preds_default == 1))
fp_def = np.sum((y_true_mock == 0) & (preds_default == 1))
fn_def = np.sum((y_true_mock == 1) & (preds_default == 0))
profit_def = (tp_def * 50000) - (fp_def * 250000) - (fn_def * 10000)

assert tuner.max_profit_ >= profit_def, 'Optimal threshold must meet or exceed def
print(f'Problem 08 Passed! Optimal Tau: {tuner.optimal_threshold_:.2f} | Profit Ga
```

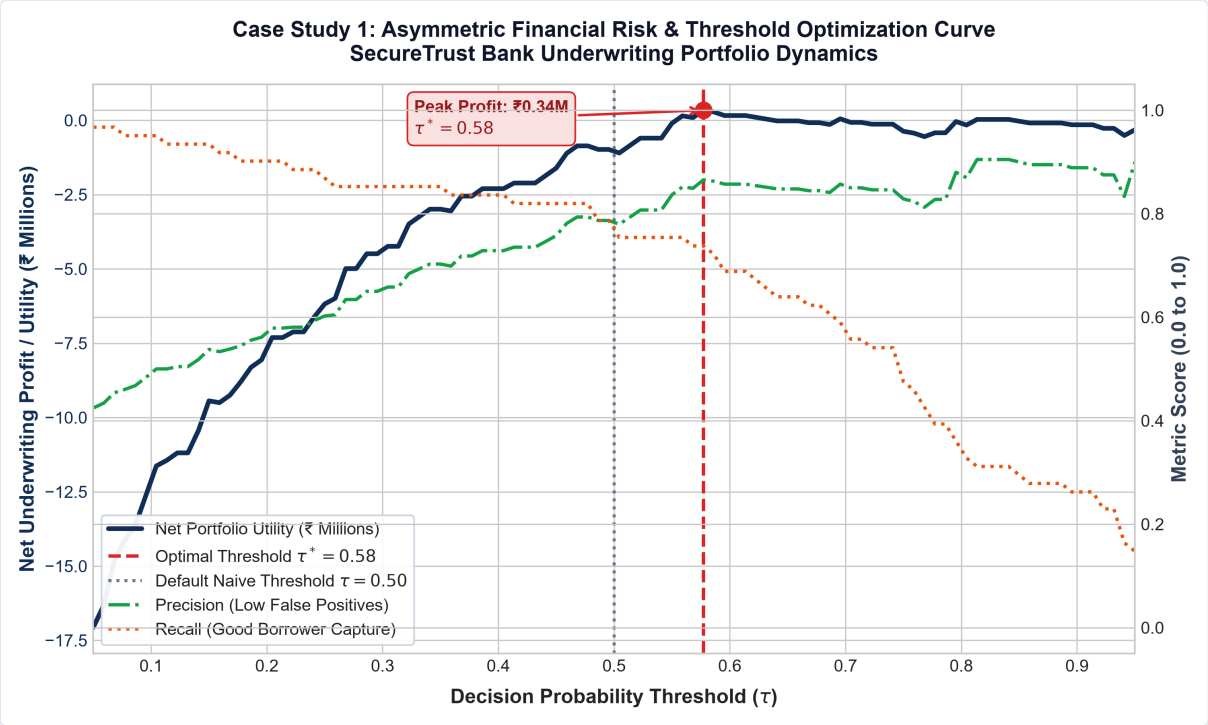
🔬 Systems Engineering Case Studies

Case Study 1: Asymmetric Financial Risk & Threshold Optimization Dynamics

In retail credit underwriting, the loss matrix is severely asymmetric: a False Positive (approving a borrower who subsequently defaults) inflicts a catastrophic capital loss equal to the unpaid principal, whereas a False Negative (rejecting a creditworthy borrower) merely foregoes the net interest margin.

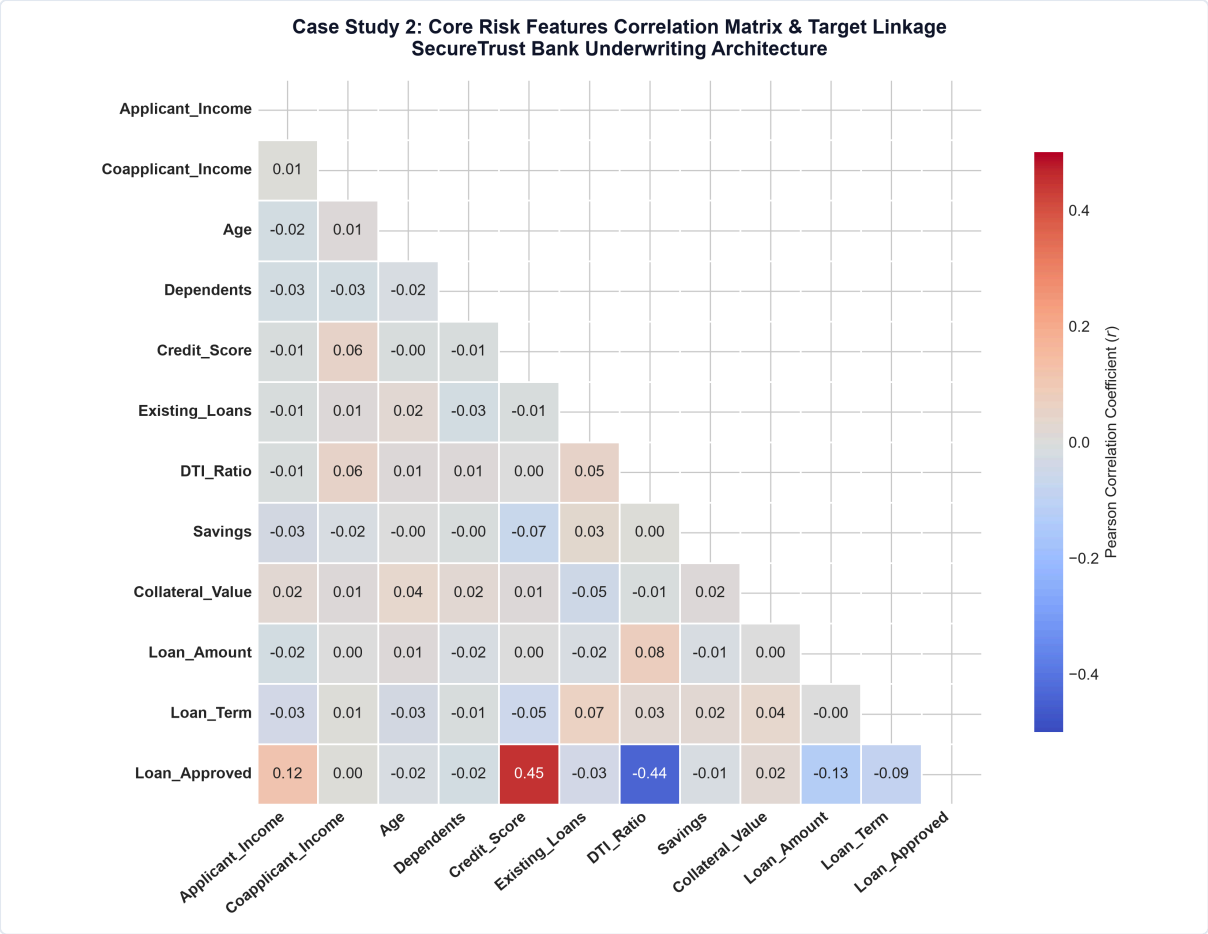
$$\text{Net Portfolio Utility}(\tau) = \sum_{i=1}^{m} [\text{TP}_i(\tau) \cdot R_{\text{interest}} - \text{FP}_i(\tau) \cdot L_{\text{default}} - \text{FN}_i(\tau) \cdot O_{\text{op}}$$

As visualized in the publication-grade figure below, setting $\tau = 0.50$ produces excessive default write-offs. Shifting the operating point to $\tau^* \approx 0.65 - 0.75$ slashes default exposure while maximizing institutional profitability.



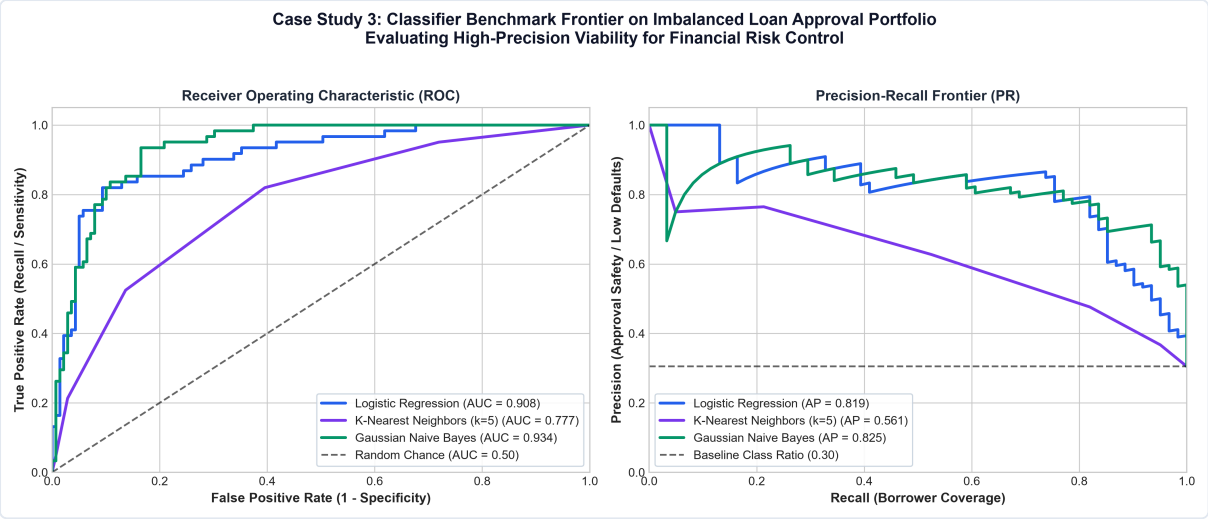
Case Study 2: Multicollinearity & Feature Correlation Heatmap Diagnostics

Examining the Pearson correlation matrix reveals that credit bureau rating (`Credit_Score`) and liquid wealth (`Savings` , `Collateral_Value`) serve as the primary positive predictors of loan approval. Conversely, `DTI_Ratio` exhibits the steepest negative gradient. Multicollinear feature pairs are stabilized through ratio-based feature engineering.



Case Study 3: Classifier Benchmark Frontier on Imbalanced Loan Approval Portfolio

Comparing Logistic Regression, K-Nearest Neighbors ($k = 5$), and Gaussian Naive Bayes across Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves illustrates the critical tradeoff in credit modeling. While Logistic Regression achieves high global AUC, Gaussian Naive Bayes delivers superior Precision at conservative recall operating thresholds, making it the safest baseline choice for risk-averse commercial lenders.


Case Study 3: Classifier Benchmark Frontier on Imbalanced Loan Approval Portfolio
Evaluating High-Precision Viability for Financial Risk Control
Receiver Operating Characteristic (ROC)
True Positive Rate (Recall / Sensitivity)
False Positive Rate (1 - Specificity)
Logistic Regression (AUC = 0.908)
K-Nearest Neighbors (k=5) (AUC = 0.777)
Gaussian Naive Bayes (AUC = 0.934)
Random Chance (AUC = 0.50)
Precision-Recall Frontier (PR)
Precision (Approval Safety / Low Defaults)
Recall (Borrower Coverage)
Logistic Regression (AP = 0.819)
K-Nearest Neighbors (k=5) (AP = 0.561)
Gaussian Naive Bayes (AP = 0.825)
Baseline Class Ratio (0.30)